Formal Methods





MCA –Software Engineering

Kantipur City College

Topics include



- Formal Methods Concept
- Formal Specification Language
  - Test plan creation
  - Test-case generation
- Executable and non-executable specifications
- Pre and Post assertions
- Formal verification

Formal methods Concept



- Formal specification is part of a more general collection of techniques that are known as 'formal methods'.
- These are all based on mathematical representation and analysis to produce consistent, complete, and correct specification of software.
- Formal methods include
  - Formal specification
  - Specification analysis and proof
  - Transformational development
  - Program verification

Acceptance of Formal methods



- Formal methods have not become mainstream software development techniques as was once predicted
  - Other software engineering techniques have been successful at increasing system quality. Hence the need for formal methods has been reduced
  - Market changes have made time-to-market rather than software with a low error count the key factor. Formal methods do not reduce time to market
  - The scope of formal methods is limited. They are not well-suited to specifying and analysing user interfaces and user interaction
  - Formal methods are hard to scale up to large systems

Use of Formal Methods



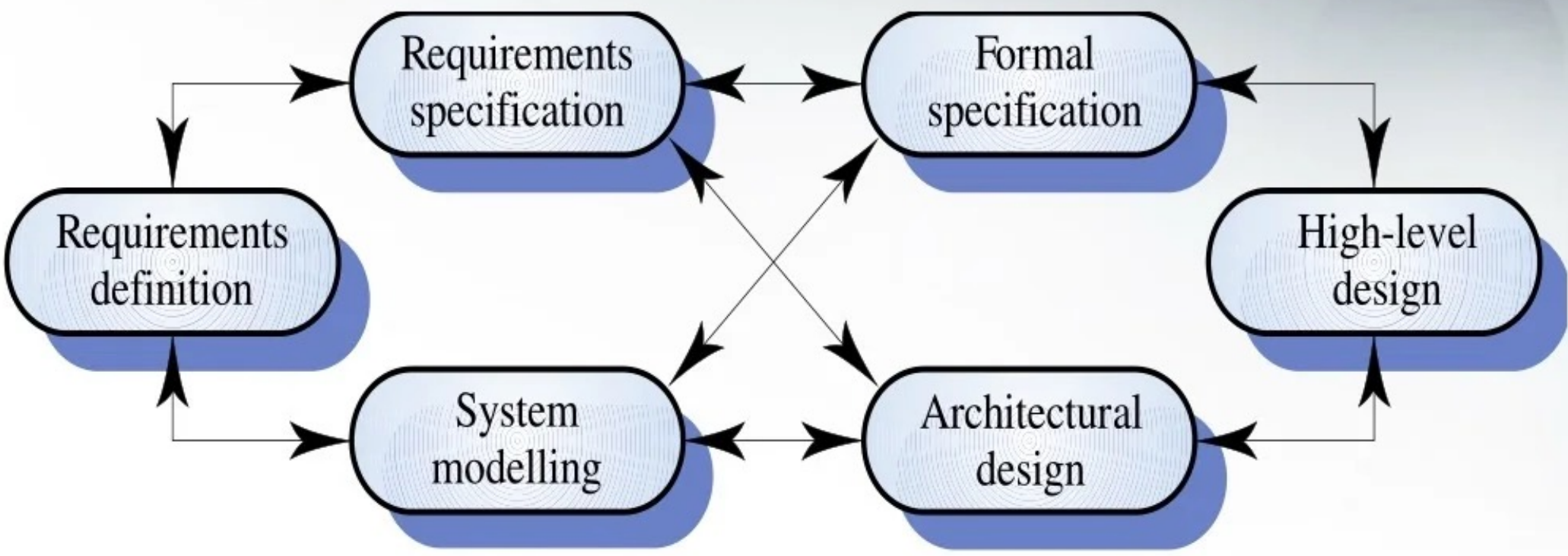
- Formal methods have limited practical applicability.
- Their principal benefits are in reducing the number of errors in systems.
- Formal specification techniques are most applicable in the development of critical systems and standards.
- In this area, the use of formal methods is most likely to be cost-effective.

Advantages of Formal Specification

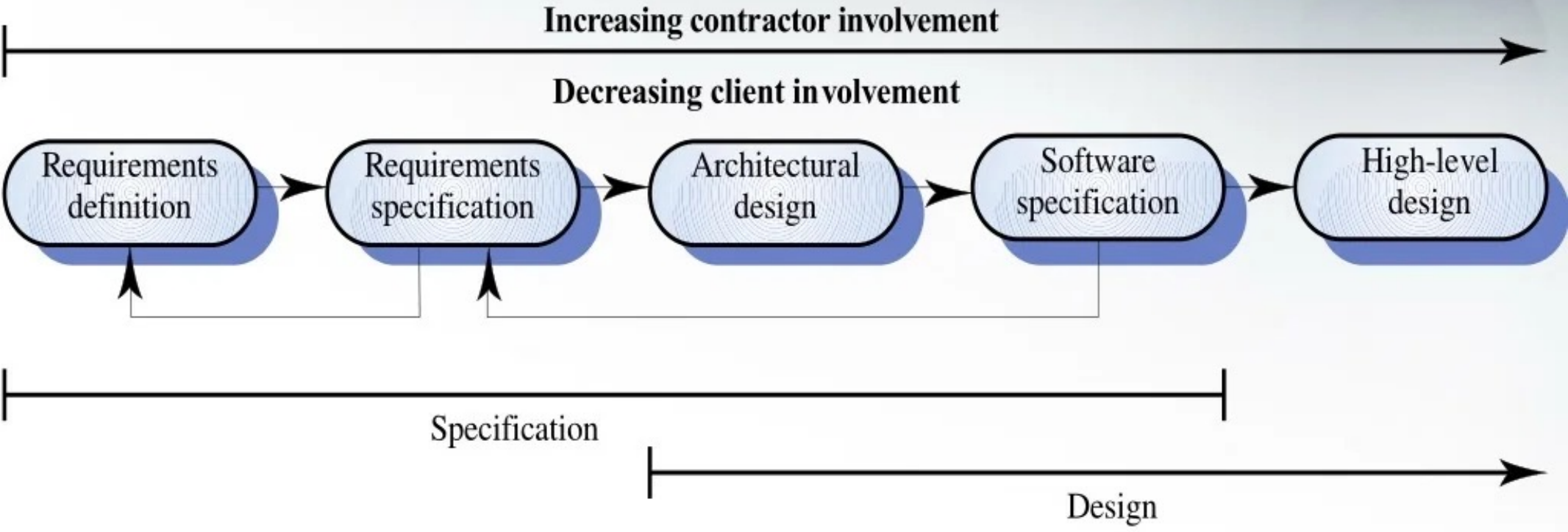


- It can be studied mathematically.
- Correctness of modules can be proved
- Equivalency can be proved.
- Incomplete definitions and inconsistencies can be detected, and
- In some cases, it may be produced automatically from requirement statements.

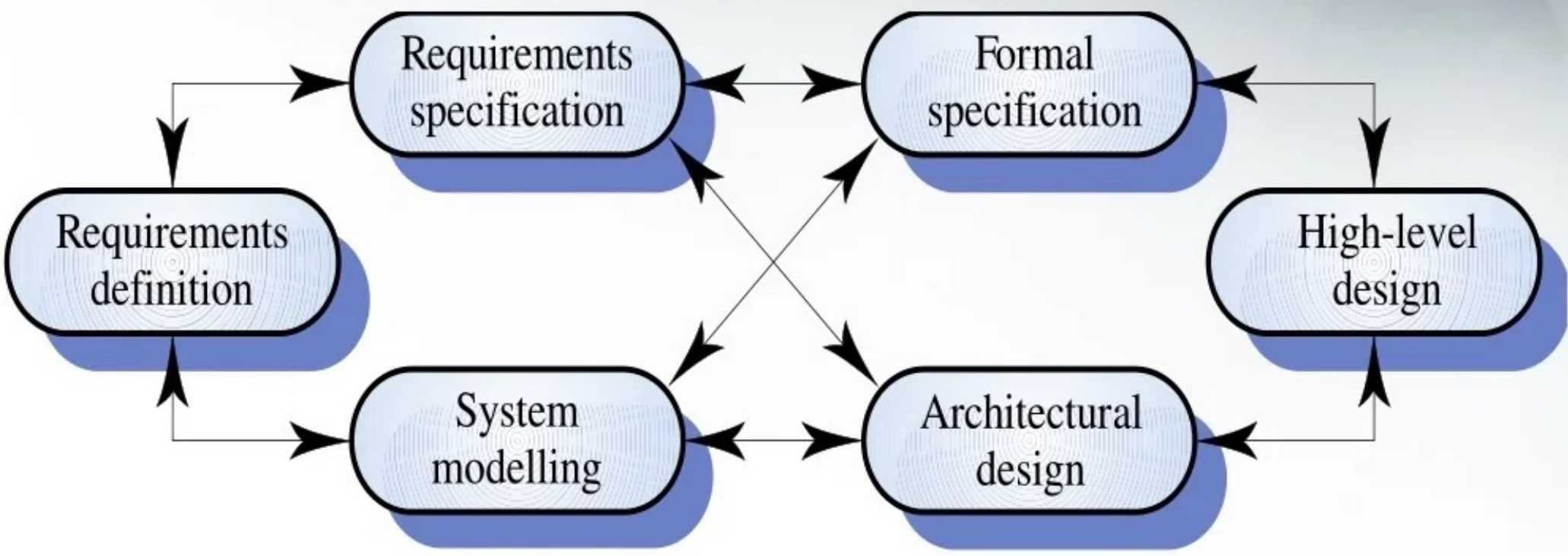
Specification in a Software Process



Specification and Design



Specification in a Software Process

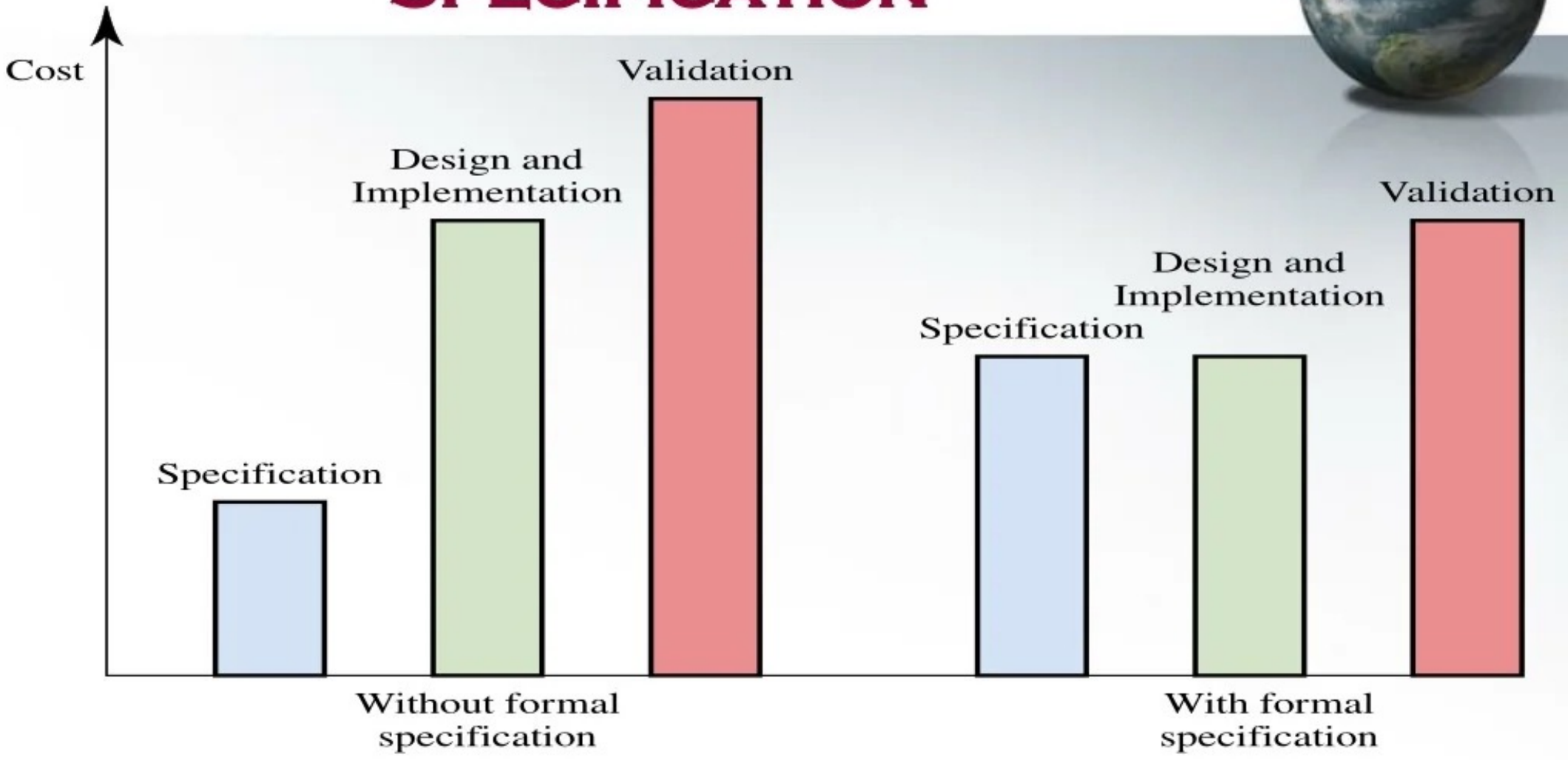


Use of Formal Specification



- Formal specification involves investing more effort in the early phases of software development
- This reduces requirements errors as it forces a detailed analysis of the requirements
- Incompleteness and inconsistencies can be discovered and resolved
- Hence, savings as much as the amount of rework due to requirements problems is reduced

Development cost of Formal Specification



Formal specification forces an analysis of the system requirements at an early stage. Correcting errors at this stage is cheaper than modifying a delivered system

Specification Techniques



- **Algebraic approach**
  - The system is specified in terms of its operations and their relationships.
  - Algebraic techniques are suited to interface specification where the interface is defined as a set of object classes.

- **Model-based approach**
  - The system is specified in terms of a state model that is constructed using mathematical constructs such as sets and sequences. Operations are defined by modifications to the system's state.

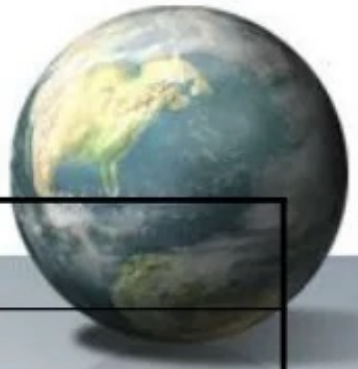
Formal Specification Languages



	Sequential	Concurrent
Algebraic	Larch (Guttag, Horning et al., 1985; Guttag, Horning et al., 1993), OBJ (Futatsugi, Goguen et al., 1985)	Lotos (Bolognesi and Brinksma, 1987),
Model-based	Z (Spivey, 1992) VDM (Jones, 1980) B (Wordsworth, 1996)	CSP (Hoare, 1985) Petri Nets (Peterson, 1981)

- A formal Specification Language are based on mathematical logic and provides for automatic logic verification.
- A formal specification may be checked for inconsistencies and contradictions before being coded in a programming language.

Mathematical Logic



Symbol	Meaning
	For all (a qualifier)
	There exists (a qualifier)
P ≡ Q	P is logically equivalent to Q
~ p	Not p
P^q	p and q
P v q	p or q
P → Q	If p then q
P ⇒ Q	P implies q
P ⟷ Q	P if and only if q
∋	Such that
P ↛ Q	P does not imply q

Examples using Logic Symbols



- x,y,z x > y^y >z ⟹ x>z

- Description: For all numeric values x,y and z for which x is larger than y and y is larger than z, x is larger than z.

Pre and Post Assertions



- A set of constraints associated with a formula are called assertions and are used to express preconditions and post-conditions for a given tasks.
- The preconditions are normally constraints placed on the input to a given formula (task), and post conditions are constraints placed on the output or results of the formula (task).
- The general format for specifying a functional task using formal specification is to define the preconditions, the process and the post conditions within the syntax and semantics of formal language being used.

Example of formal specification



- **Example 3**: The following are the precondition and post-condition for a function that is meat to sort an array of positive integers.
- **Definition**:

$\{ n>0, \forall i\ (0<i \leq n) \rightarrow (a[i] > 0 \ \}$ Precondition

Program to soft array a[1,…n]

$\{\forall i\ (0<i<n) \rightarrow a[i] \leq a[i+1]\}$ Post-condition

Description: These equations mean that before the task is performed we have an array of positive integers, after the task is performed we have the same array of positive integers, and the content of the array is in ascending order.

Example of formal specification



- **Example 3**: The following are the precondition and post-condition for a function that is meat to sort an array of positive integers.
- **Definition**:

$\{ n>0, \forall i\ (0<i \leq n) \rightarrow (a[i] > 0 \ \}$ Precondition

Program to soft array a[1,…n]

$\{\forall i\ (0<i<n) \rightarrow a[i] \leq a[i+1]\}$ Post-condition

Description: These equations mean that before the task is performed we have an array of positive integers, after the task is performed we have the same array of positive integers, and the content of the array is in ascending order.

Example of formal specification



- **Example 3**: The following are the precondition and post-condition for a function that is meat to sort an array of positive integers.
- **Definition**:

$\{ n>0, \forall i\ (0<i \leq n) \rightarrow (a[i] > 0 \ \}$ Precondition

Program to soft array a[1,…n]

$\{\forall i\ (0<i<n) \rightarrow a[i] \leq a[i+1]\}$ Post-condition

Description: These equations mean that before the task is performed we have an array of positive integers, after the task is performed we have the same array of positive integers, and the content of the array is in ascending order.

Behavioural Specification



- Algebraic specification can be cumbersome when the object operations are not independent of the object state
- Model-based specification exposes the system state and defines the operations in terms of changes to that state
- The Z notation is a mature technique for model-based specification. It combines formal and informal description and uses graphical highlighting when presenting specifications

References



- From software engineering, A practitioner's approach by Roger S. Pressman
  - Chapter 25: Formal Methods
    - Basic concepts, deficiencies of Less formal approaches,
    - Formal methods concept, Mathematical preliminaries
    - Formal specification languages
    - Summary of Z Notation.
- From Software Engineering, Ian Sommerville
  - Part5: Verification and Validation
    - Chapter 9: Formal Specification
    - Chapter 21: Critical System Validation
- From Software Engineering Fundamentals by Ali Behforooz and F.J. Hudson
  - Chapter 5: Software Specification Tools